SASE / SSE Architecture & Design Blueprint

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — SASE / SSE Architecture & Design Blueprint 4

The situation . . . 4

The solution . . . 4

What you get . . . 4

The standard deployment model . . . 5

The outcome . . . 5

1. Solution Overview — SASE / SSE Architecture & Design Blueprint 5

What this solution is . . . 5

Why piecemeal edge security fails (the *why*) . . . 6

The capability services and how they fit . . . 6

Identity as the control plane . . . 7

The phased adoption roadmap . . . 7

The target operating model . . . 8

Benefits . . . 8

Honest limitations . . . 8

The standard deployment model . . . 9

2. Requirements Guide — SASE / SSE Architecture & Design Blueprint 9

R1. Netskope tenant with the licensed modules for the in-scope capabilities *(GATING)* . . . 9

R2. Executive sponsorship and a named program owner *(GATING)* . . . 10

R3. Microsoft Entra ID as the identity control plane (SSO + groups) . . . 10

R4. Endpoint management to deliver the client and certificates . . . 10

R5. Network & egress design inputs (steering decision + exceptions) . . . 11

R6. Current-state edge inventory (the design baseline) . . . 11

R7. The named architecture artifacts exist (design deliverables) . . . 11

R8. Phase-scope targeting groups (Entra security groups) . . . 11

R9. SSL-inspection trust + governed exception model . . . 12

R10. ZTNA private-app inputs (for the ZTNA phase) . . . 12

R11. Change control, rollback, and a comms plan (per phase) . . . 12

R12. KPI baseline + operating-model owners (for the run phase) . . . 13

Checklist . . . 13

3. Implementation Guide — SASE / SSE Architecture & Design Blueprint 14

Part A — Design artifacts (Configure-phase deliverables) . . . 14

Part B — Target operating model: `SASE Operating Model – Edge – Run` . . . 15

Part C — Phased adoption roadmap (execution) . . . 16

Part D — Transition to run . . . 18

5. Testing & Acceptance — SASE / SSE Architecture & Design Blueprint 18
Group 0 — Design & operating-model readiness (run once, before Phase 0 expands) 19
Group 1 — Phase 0: Baseline & Steering 19
Group 2 — Phase 1: Secure Web Gateway (SWG) 19
Group 3 — Phase 2: CASB & Cloud-App Control 20
Group 4 — Phase 3: ZTNA for Private Apps 20
Group 5 — Phase 4: Data-Protection-Adjacent & Optimization 20
Group 6 — Operating-model health (ongoing) 21
Program-level exit (per phase) 21
6. Troubleshooting Guide — SASE / SSE Architecture & Design Blueprint 21
A. Steering / client not active 21
B. SSL inspection / certificate trust 22
C. Identity / policy not behaving as designed 22
D. CASB / cloud-app control 23
E. ZTNA / VPN displacement 23
F. Phasing / gate violations 23
G. Design / operating-model issues 24
H. Tooling / script issues 24
When to escalate 24
7. Best Practices — SASE / SSE Architecture & Design Blueprint 25
Design discipline 25
Phasing & the pilot gate 25
SSL inspection 25
ZTNA & VPN displacement 26
CASB & shadow IT 26
Data-protection-adjacent scope 26
Change control & rollback 26
Operating model 26
8. Deliverables Summary — SASE / SSE Architecture & Design Blueprint 27
Documentation 27
PowerShell scripts 28
Hero image 29
What the buyer receives 29
Outcome 29
Appendix A — PowerShell Scripts 30

Executive Summary — SASE / SSE Architecture & Design Blueprint

The situation

Most organizations did not *decide* on an edge-security architecture — they accumulated one. A secure web gateway was bought to satisfy an audit finding; a Zero Trust access tool was added when VPN buckled; a cloud-app discovery product arrived with a different team; SSL inspection was switched on in one place and not another. The result is a set of overlapping, inconsistently configured edge controls that are expensive to license, hard to operate, and impossible to reason about as a whole. Policy lives in several consoles, identity is wired in differently to each, and nobody can answer the simple question: *for this user, on this device, going to this destination, which control decides, and why?*

The remedy is not another point product. It is a **coherent reference architecture** that ties the edge services together under one model, with **identity as the control plane**, a **phased adoption roadmap** so the move is staged rather than disruptive, and a **target operating model** that names who runs it and how success is measured.

The solution

This blueprint delivers exactly that — a repeatable, vendor-aligned design for a Secure Access Service Edge / Security Service Edge estate that unifies five capability areas:

- **Secure Web Gateway (SWG)** — inline inspection and policy for web and SaaS traffic.
- **Zero Trust Network Access (ZTNA)** — identity- and device-aware access to private applications, replacing broad VPN network access.
- **Cloud Access Security Broker (CASB)** — visibility and control over sanctioned and unsanctioned cloud applications (inline and, where applicable, API-based).
- **SSL/TLS inspection** — the decryption foundation that makes the other controls effective, with a governed exception model.
- **Data-protection-adjacent controls (DLP-adjacent)** — content-aware policy hooks positioned in the architecture (the deep DLP build is a separate program; here it is *designed in*, not detailed).

These are bound together by **identity** (Microsoft Entra ID) as the single control plane for who a user is, what device they are on, and which policy applies — and delivered through a phased roadmap proven on a **pilot phase** before any expansion.

What you get

- A plain-language **Solution Overview** of the SASE/SSE model and *why* it beats piecemeal edge security.
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite (Netskope tenant and modules, Entra ID identity, endpoint management, network/egress inputs, executive sponsorship).

• An **Implementation Guide** that names the **reference architecture**, the **policy model**, the

roadmap phases, and the **operating-model roles**, then walks Assess → Configure → Pilot → Validate → Expand with an explicit pilot-phase gate.

• Five **PowerShell scripts** for current-state edge/identity discovery, architecture-readiness

validation, a capability-coverage and gap report, safe remediation of configuration gaps, and a phase-scope group builder.

• **Testing** and **Troubleshooting** guides with pass/fail gates, plus field-proven **Best Practices**.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with a hard pilot-phase gate before expansion.

The outcome

One coherent SASE/SSE architecture — SWG, ZTNA, CASB, SSL inspection, and identity-based policy designed as a single operating model with a phased roadmap — replacing piecemeal edge security, with a named operating model and measurable KPIs, proven on a pilot phase before you scale.

1. Solution Overview — SASE / SSE Architecture & Design Blueprint

What this solution is

This is a **reference architecture and design blueprint** for a SASE/SSE estate — not a single product configuration and not a one-button wizard. It is a repeatable design that ties the edge security services together into one coherent model, with a **phased adoption roadmap** and a **target operating model**, so an organization can roll out every edge service against the same blueprint rather than buying and operating them in silos.

Concretely, the blueprint delivers three durable artifacts that the rest of this package builds and operates:

1. A named **reference architecture** — `SASE Reference Architecture – Edge – Design` — that shows how

the capability services fit together and how traffic and identity flow through them.

1. A named **policy model** — `SASE Policy Model – Edge – Design` — a layered way of writing policy

(identity-first, then destination/app, then content) that is consistent across every service.

1. A named **adoption roadmap** — `SASE Adoption Roadmap – Edge – Program` — a sequence of phases that

moves the estate from piecemeal to unified, each with an entry and a gate.

It also defines a **target operating model** — `SASE Operating Model – Edge – Run` — with roles, RACI, and KPIs so the design is *run*, not just *built*.

Why piecemeal edge security fails (the *why*)

Edge controls bought separately almost always end up:

• **Overlapping** — two products inspect the same traffic, or two access paths exist to the same app, doubling cost and confusion.

• **Inconsistent** — SSL inspection is on in one path and off in another; policy is identity-aware in one console and IP-based in the next; the same user gets different treatment depending on which tool is in front.

• **Un-auditable** — no single place answers "which control decided, and why," because the decision is spread across consoles with different data models.

• **Hard to operate** — every product has its own admin model, its own identity wiring, its own reporting, and its own upgrade cadence.

A SASE/SSE architecture fixes this by making the edge **one logical service** with **identity as the control plane**: who the user is and what device they are on is evaluated *once* and drives policy *everywhere* — SWG, ZTNA, CASB, and inspection. That single shift — from network-location trust to identity-and-device trust, evaluated consistently at the edge — is the whole point of SASE/SSE.

The capability services and how they fit

The blueprint composes five capability areas. Each is a layer of the same edge, not a separate island. Exact Netskope console screens for configuring each are **deferred to the official Netskope documentation and your tenant** (see the vendor note in `global-requirements-network.md`); this section is about *what each does and where it sits in the architecture*.

Capability	What it does	Where it sits in the architecture
Secure Web Gateway (SWG)	Inline inspection and policy for web/SaaS traffic (URL/category filtering, threat protection, acceptable-use).	The inline path for internet/SaaS-bound traffic; consumes identity from the control plane.
Zero Trust Network Access (ZTNA)	Identity- and device-aware access to *private* applications; replaces broad VPN network access with per-app, least-privilege access.	The private-app path; brokers connections by identity, not network location.
Cloud Access Security Broker (CASB)	Visibility into and control over sanctioned/unsanctioned cloud apps; inline controls and (where supported) API-based posture.	Rides the inline path (and API connectors) to govern cloud-app usage and shadow IT.
SSL/TLS inspection	Decrypts inspectable traffic so SWG/CASB/threat controls can actually see content, with a governed exception list.	The decryption foundation under the inline path; everything inline depends on it.

Capability	What it does	Where it sits in the architecture
Data-protection-adjacent (DLP-adjacent)	Content-aware policy hooks (where to apply data classification/blocking) positioned in the design.	Designed into the inline and CASB layers; the deep DLP build is a separate program.

> ***Scope honesty:*** *this blueprint *designs in* the DLP-adjacent control points (where data-aware policy will attach) but does not build a full data-classification/DLP program — that is a separate engagement. Naming the hooks now means the later DLP work drops into the architecture cleanly instead of bolting on.*

Identity as the control plane

In this design, **Microsoft Entra ID is the control plane**. The edge does not decide policy from IP addresses or network segments; it decides from **identity and device posture**:

- **SSO and identity** — users authenticate via Entra ID; the edge trusts that assertion.
- **Groups drive steering and policy** — phase scope, ring membership, and policy targeting all key off named Entra security groups (built by the supplied `SASE-Set-PhaseScopeTargeting.ps1`).
- **Device posture** — managed/compliant device signals (from Intune) feed access decisions (especially ZTNA).
- **One decision, applied everywhere** — the same identity context drives SWG, ZTNA, and CASB policy,

which is what makes the estate *coherent* rather than five separate trust models.

The phased adoption roadmap

The blueprint is adopted in **named phases** (`SASE Adoption Roadmap – Edge – Program`), each a deliberate step from piecemeal to unified. The phases are summarized here and detailed in the Implementation Guide:

- **Phase 0 — Baseline & Steering** — establish identity wiring, deploy the client/steering to a small scope, turn on visibility (monitor-only).
- **Phase 1 — Secure Web Gateway** — bring web/SaaS traffic under one SWG policy with SSL inspection and a governed exception model.
- **Phase 2 — CASB & Cloud-App Control** — add sanctioned/unsanctioned cloud-app visibility and inline control; tame shadow IT.
- **Phase 3 — ZTNA for Private Apps** — publish private apps via identity-aware access and begin VPN displacement.
- **Phase 4 — Data-Protection-Adjacent & Optimization** — activate the designed-in content-aware hooks

and tune the estate.

Each phase runs the standard model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, and **no phase expands until its pilot passes the gate**.

The target operating model

A design nobody runs decays. The blueprint ships a **target operating model** (`SASE Operating Model – Edge – Run`) so the estate has named owners and measurable health:

- **Roles** — `SASE Architecture Owner – Edge – Run`, `SASE Policy Administrator – Edge – Run`,

`SASE Operations Engineer – Edge – Run`, `SASE Identity Liaison – Edge – Run`, and an executive `SASE Sponsor – Program – Run` (detailed in the Implementation Guide).

- **KPIs** — steering coverage, SSL-inspection coverage (with a tracked exception rate), policy

consistency, ZTNA/VPN displacement, mean time to policy change, and user-impact/ticket rate.

- **Cadence** — a regular policy-review and exception-review rhythm, plus a phase-gate review.

Benefits

- **One coherent edge** — SWG, ZTNA, CASB, and inspection operated as a single, identity-driven service.
- **Lower cost and overlap** — designing first surfaces duplicate tools and paths before more are bought.
- **Auditable decisions** — a single policy model means you can explain why any decision was made.
- **De-risked rollout** — phased adoption with a pilot gate per phase; live traffic is never changed

blindly.

- **A run model, not just a build** — named roles and KPIs keep the architecture healthy over time.

Honest limitations

- **This is a design + adoption blueprint, not a turnkey tenant build.** It names the architecture,

policy model, roadmap, and operating model and walks the rollout; the **exact Netskope console configuration is deferred to Netskope's official documentation and your tenant**.

- **It requires real prerequisites** — a licensed Netskope tenant and modules, Entra ID identity,

endpoint management to deliver the client/certificates, network/egress inputs, and executive sponsorship (see Requirements).

- **Deep DLP is out of scope** — the architecture *designs in* the content-aware hooks; building a full

data-classification/DLP program is a separate engagement.

- **SSL inspection has a governed exception model** — some destinations (e.g. regulated, pinned, or

privacy-sensitive categories) are inspected by exception; coverage is a tracked KPI, not 100%.

• **It is phased, not same-day** — a piecemeal-to-unified move is deliberately staged, wave by wave, behind change control.

The standard deployment model

This solution follows the **standard deployment model** at two levels: the **program** is delivered in roadmap phases, and **each phase** internally follows the same lifecycle —

Assess → **Configure** → **Pilot** → **Validate** → **Expand**, with a **pilot-phase gate** that must pass before any phase expands to broader scope.

2. Requirements Guide — SASE / SSE Architecture & Design Blueprint

Verify **every** requirement before configuring. Each entry is structured as **Description · Why · Verify · Implement-if-missing. Never assume a prerequisite exists.** Category-wide prerequisites (Netskope tenant, Entra ID, Intune, admin roles, network/identity fundamentals, pilot group, change control) are in `global-requirements-network.md`; this guide adds the requirements specific to a SASE/SSE *architecture* engagement and numbers them R1–R12.

> ***Naming convention (mandatory, reused verbatim throughout):*** *every artifact is named* `<Purpose> – <Scope> – <Stage>`*, e.g.* `SASE Reference Architecture – Edge – Design`*. The canonical names introduced here are reused in the Implementation, Testing, and Troubleshooting guides.*
>
> ***Vendor note:*** *exact Netskope console screens are* ***deferred to the official Netskope documentation and your tenant****. Requirements below name *what must exist*, not which menu to click.*
>
> ***Gating prerequisites:*** *R1 (Netskope tenant + licensed modules) and R2 (executive sponsorship) are* ***hard gates*** *— no architecture work proceeds past the Assess phase until both are satisfied.*

R1. Netskope tenant with the licensed modules for the in-scope capabilities *(GATING)*

• **Description:** An active Netskope tenant licensed for the capability services this blueprint will adopt — at minimum SWG, plus ZTNA (Private Access), CASB, and SSL inspection as the roadmap dictates.

• **Why:** The tenant is the edge control plane (global requirement G1). Each roadmap phase needs the corresponding module licensed; an unlicensed module cannot be designed into a phase that ships.

• **Verify:** Sign in to the Netskope admin console as an administrator (G1); confirm the in-scope modules are licensed/entitled for the tenant. Run `SASE-Discovery.ps1` to record the current-state edge/identity inputs the design depends on.

• **Implement-if-missing:** Engage your Netskope account team / reseller to license the required modules sized to the roadmap; do not commit a phase to a module the tenant is not entitled to.

R2. Executive sponsorship and a named program owner *(GATING)*

• **Description:** An executive sponsor (SASE `Sponsor – Program – Run`) and a named architecture owner

(SASE `Architecture Owner – Edge – Run`) accountable for the program.

• **Why:** A SASE/SSE move changes how every user reaches the internet and private apps and crosses network, identity, and endpoint teams. Without sponsorship and a single owner, phases stall and exceptions multiply. This is an architecture/operating-model requirement, not a technical one.

• **Verify:** Confirm a documented sponsor and architecture owner with authority over the roadmap and a budget/decision mandate.

• **Implement-if-missing:** Secure executive sponsorship and assign the architecture owner before any build; record both in the operating model (§03).

R3. Microsoft Entra ID as the identity control plane (SSO + groups)

• **Description:** Entra ID configured as the identity source for SSO to the edge, with the security groups that will drive phase scope and policy targeting.

• **Why:** The whole design rests on **identity as the control plane** — policy and steering key off identity and group membership, not network location (global requirements G2, G5).

• **Verify:** Sign in to `https://entra.microsoft.com` (G2); confirm SSO/identity federation to the edge is configured and that users/groups are present. Run `SASE-Validation.ps1` to confirm the required targeting groups exist.

• **Implement-if-missing:** Configure SSO between Entra ID and the edge per the vendor's current guidance; create the targeting groups with `SASE-Set-PhaseScopeTargeting.ps1` (R8).

R4. Endpoint management to deliver the client and certificates

• **Description:** Microsoft Intune (or equivalent) able to deploy the edge client, the root/decryption certificate(s), and the steering configuration to in-scope endpoints.

• **Why:** SSL inspection requires the decryption root certificate to be trusted on every steered endpoint; steering and the client must be delivered and kept current at scale (global requirement G3).

• **Verify:** `https://intune.microsoft.com` loads (G3); confirm target devices are enrolled/compliant

and that a deployment channel for the client + certificate exists. `SASE-Discovery.ps1` reports enrolled/compliant device counts as a readiness input.

• **Implement-if-missing:** Stand up the deployment (Intune app + certificate profile + steering config) per the vendor's current packaging guidance; do not enable inline SSL inspection before the decryption certificate is trusted on the steered scope.

R5. Network & egress design inputs (steering decision + exceptions)

• **Description:** Documented network inputs: how traffic will be steered (client-based vs. network-based), egress/proxy topology, and the initial **SSL-inspection exception** candidates (pinned-cert apps, regulated/privacy-sensitive categories).

• **Why:** Inline inspection and ZTNA change live traffic flow; the steering method and a governed exception list must be decided in design, not discovered in production (global requirements G5, G7).

• **Verify:** Confirm a written steering decision and a first-draft exception list captured in the reference architecture (`SASE Reference Architecture – Edge – Design`). Endpoints resolve DNS and reach the internet (G5).

• **Implement-if-missing:** Author the steering decision and the exception model as part of the Assess phase; record both in the reference architecture before any inline enforcement.

R6. Current-state edge inventory (the design baseline)

• **Description:** An inventory of the existing edge controls and identity inputs the design must account for — current proxy/SWG, any VPN/remote-access, any cloud-app discovery/CASB, current SSL inspection state, and the Entra identity/groups picture.

• **Why:** You cannot design a *unified* edge without knowing what overlapping controls and paths already exist; the baseline drives the gap analysis and the roadmap sequencing.

• **Verify:** Confirm a baseline exists (output of `SASE-Discovery.ps1` — a read-only inventory of edge and identity current state) reviewed by the architecture owner.

• **Implement-if-missing:** Run `SASE-Discovery.ps1` (read-only) and capture any non-discoverable context (existing third-party SWG/VPN/CASB) manually into the baseline.

R7. The named architecture artifacts exist (design deliverables)

• **Description:** The three core design artifacts are authored and version-controlled: the reference architecture, the policy model, and the adoption roadmap.

• **Why:** These are the deliverables the entire program executes against; without them, "SASE" degrades back into piecemeal tool config.

• **Verify:** Confirm `SASE Reference Architecture – Edge – Design`, `SASE Policy Model – Edge – Design`, and `SASE Adoption Roadmap – Edge – Program` exist and are approved.

• **Implement-if-missing:** Produce them in the Configure phase (§03) following the templates and the naming convention; have the architecture owner approve them.

R8. Phase-scope targeting groups (Entra security groups)

• **Description:** Named Entra security groups that scope each roadmap phase and its pilot, e.g. `SASE Phase Scope – SWG – Pilot`, `SASE Phase Scope – SWG – Broad`, and equivalents per phase.

• **Why:** Phased adoption and the pilot gate depend on being able to target exactly the right users/devices per phase; identity-driven policy needs these groups to exist (global requirement G6).

• **Verify:** Entra → **Groups**; confirm the per-phase pilot/broad groups exist (or run `SASE-Validation.ps1`). The pilot group requirement is global requirement G6.

• **Implement-if-missing:** Build the groups with `SASE-Set-PhaseScopeTargeting.ps1` (defaults to `-WhatIf`; re-run with `-Apply` to create), named per the convention.

R9. SSL-inspection trust + governed exception model

• **Description:** The decryption root certificate is trusted on every steered endpoint, and a governed, documented **exception list** exists for traffic that must not be inspected.

• **Why:** Inspection is the foundation the inline controls depend on; without trusted certificates users get TLS errors, and without a governed exception model you break pinned/regulated apps.

• **Verify:** On a steered endpoint, confirm the decryption root is in the trusted store (browsers/OS trust the edge certificate); confirm the exception list is recorded in the reference architecture. `SASE-Validation.ps1` checks certificate-trust readiness signals.

• **Implement-if-missing:** Deploy the root certificate via R4; author and approve the exception list as part of `SASE Reference Architecture – Edge – Design` before enabling inline inspection.

R10. ZTNA private-app inputs (for the ZTNA phase)

• **Description:** A catalog of the private applications to publish via ZTNA — FQDNs/ports, the identity/groups that should reach each, and the connector/placement decision.

• **Why:** ZTNA replaces broad VPN access with per-app least-privilege access; you must know which apps, for whom, before publishing — and VPN displacement must be planned, not abrupt.

• **Verify:** Confirm a private-app catalog mapped to Entra groups exists for the ZTNA phase (`SASE Phase Scope – ZTNA – Pilot` / `– Broad`).

• **Implement-if-missing:** Inventory private apps and owners; map each to its access group; record in the roadmap's ZTNA phase. (Connector placement specifics are deferred to Netskope documentation.)

R11. Change control, rollback, and a comms plan (per phase)

• **Description:** An approved change window, a documented rollback (disable steering / remove client / revert policy / re-enable VPN path), and a user-comms plan — for **each** phase cutover.

• **Why:** Every phase changes live traffic (inline enforcement, decryption, VPN displacement). CMMC-grade operational safety and global requirement G7 require approved windows and a tested back-out.

• **Verify:** Confirm an approved change record and a written rollback per planned phase, plus a comms template (global requirement G7).

• **Implement-if-missing:** Establish the change/comms process; author per-phase rollback runbooks (covered in Best Practices §07).

R12. KPI baseline + operating-model owners (for the run phase)

• **Description:** A baseline of the operating-model KPIs (steering coverage, inspection coverage/ exception rate, policy consistency, ZTNA/VPN displacement, mean time to policy change, user-impact rate) and the named operating-model roles assigned.

• **Why:** A design that nobody measures or owns decays; the operating model (`SASE Operating Model –
Edge – Run`) needs a starting baseline and named owners to govern the estate over time.

• **Verify:** Confirm baseline KPI values (from `SASE-Report.ps1` plus manual inputs) and that the operating-model roles (§03) are assigned.

• **Implement-if-missing:** Capture the KPI baseline early (during Assess/Pilot) and assign the operating-model roles before the first phase expands.

Checklist

• **(GATING)** Netskope tenant + licensed in-scope modules — `SASE-Discovery.ps1`

• **(GATING)** Executive sponsor + named architecture owner

• Entra ID SSO + targeting groups present — `SASE-Validation.ps1`

• Endpoint management delivers client + decryption certificate

• Steering decision + SSL-inspection exception list drafted

• Current-state edge/identity baseline captured — `SASE-Discovery.ps1`

• Reference architecture, policy model, adoption roadmap authored & approved

• Per-phase pilot/broad targeting groups — `SASE-Set-PhaseScopeTargeting.ps1`

• Decryption root trusted on steered endpoints + governed exception model

• ZTNA private-app catalog mapped to groups (for the ZTNA phase)

• Per-phase change control, rollback, comms plan

• KPI baseline captured + operating-model roles assigned — `SASE-Report.ps1`

3. Implementation Guide — SASE / SSE Architecture & Design Blueprint

This is the **program runbook**. It produces the three design artifacts, defines the operating model, and then executes the phased adoption roadmap. The program follows the standard model at two levels: the **roadmap** is a sequence of phases, and **each phase** internally runs **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a **pilot-phase gate** that must pass before the phase expands.

> ***Naming convention (mandatory, reused verbatim):*** *every artifact, phase, role, and group is named* `<Purpose> – <Scope> – <Stage>`*. Stage is* `Design` *(an architecture artifact),* `Program` *(a roadmap/ cross-cutting item),* `Pilot` */* `Broad` *(a rollout scope), or* `Run` *(an operating-model item). The canonical names introduced here are reused in Testing (§05) and Troubleshooting (§06).*

> ***Vendor note (read first):*** *the* ***exact Netskope console*** *screens, menu paths, and tenant options are* ***deferred to the official Netskope documentation and your tenant****. Steps below say *what* to configure and *in what order*; confirm the precise control in your console. Identity/edge current-state discovery uses* ***Microsoft Graph PowerShell*** *(the supplied scripts), which is real and current.*

Part A — Design artifacts (Configure-phase deliverables)

Before any traffic is touched, author the three artifacts. They are the things the whole program executes against (requirement R7).

A1 — Reference architecture: `SASE Reference Architecture – Edge – Design`

Define, on one page plus detail, how the capability services compose into one edge:

1. **Identity control plane.** Entra ID is the source of truth; SSO to the edge; group membership and device posture (from Intune) feed every policy decision. State explicitly: *policy is decided from identity + device, not network location.*
1. **Inline path.** The steered traffic flow: endpoint → steering (client-based or network-based, per R5) → SSL inspection (with the governed exception list, R9) → SWG policy → CASB inline controls → internet/SaaS.
1. **Private-app path.** ZTNA brokers identity-aware, per-app access to private applications (R10), displacing broad VPN network access.
1. **DLP-adjacent hooks.** Mark *where* content-aware policy will attach (inline and CASB layers) so the later DLP program drops in — do not build DLP here.
1. **Exception model.** The SSL-inspection exception list and the steering-bypass list, both governed and recorded here.
1. **Trust boundaries.** Note what is decrypted, what is bypassed, and where regulated/privacy-sensitive traffic is handled.

Output: an approved SASE `Reference Architecture – Edge – Design` document (diagram + narrative).

A2 — Policy model: SASE Policy Model – Edge – Design

Define **one** way to write policy, applied consistently across SWG, ZTNA, and CASB. The model is layered, evaluated in this order:

1. **Identity layer (first).** Who is the user / which Entra group, and what is the device posture?

Everything keys off this.

1. **Destination / application layer.** Web category/URL, SaaS app instance, or named private app.
2. **Content layer (DLP-adjacent hook).** Where content-aware rules will attach (designed in now).
3. **Action.** Allow / inspect / restrict / block, plus user-coaching where the capability supports it.

State the conventions: least-privilege default, identity-first targeting, named policy objects using the convention (e.g. `SASE Policy – SWG Acceptable Use – Broad`), and a change/exception process.

Output: an approved `SASE Policy Model – Edge – Design`.

A3 — Adoption roadmap: SASE Adoption Roadmap – Edge – Program

Sequence the phases (Part C), each with: entry criteria, the targeting groups it uses, its pilot scope, its acceptance gate (§05), and its rollback (R11). Record it as `SASE Adoption Roadmap – Edge – Program`.

Part B — Target operating model: SASE Operating Model – Edge – Run

A design must be *run*. Define the operating model before the first phase expands (requirement R12).

B1 — Roles (RACI)

Role (named)	Responsibility
`SASE Sponsor – Program – Run`	Executive accountability, budget, cross-team mandate, phase-gate sign-off.
`SASE Architecture Owner – Edge – Run`	Owns the reference architecture and policy model; approves changes; final say on exceptions.
`SASE Policy Administrator – Edge – Run`	Authors/maintains policy per the policy model; runs the exception process.
`SASE Operations Engineer – Edge – Run`	Day-to-day edge operations, steering/client health, incident response, validation runs.
`SASE Identity Liaison – Edge – Run`	Owns the Entra group/targeting wiring and device-posture signals; bridges identity + edge teams.

B2 — KPIs (the run dashboard)

KPI	What it measures	Target posture
Steering coverage	% of in-scope endpoints actively steered	Trend to ~100% of the active phase scope

KPI	What it measures	Target posture
SSL-inspection coverage	% of inspectable traffic decrypted	High, with the **exception rate tracked** (not chased to 100%)
Policy consistency	% of traffic governed by the unified policy model (vs. legacy point tools)	Increases each phase as silos retire
ZTNA / VPN displacement	% of private-app access via ZTNA vs. legacy VPN	Increases through the ZTNA phase
Mean time to policy change	Time from request to safely-applied policy	Decreases as the model matures
User-impact / ticket rate	Edge-related tickets per active user	Low and stable; spikes trigger a review

B3 — Cadence

- **Policy review** (regular): new/changed policy against the policy model.
- **Exception review** (regular): SSL-inspection and steering-bypass exceptions — re-justify or retire.
- **Phase-gate review** (per phase): the pilot acceptance gate (§05) before expansion.

Part C — Phased adoption roadmap (execution)

Each phase below runs the standard model. **The pilot-phase gate is mandatory: a phase does not expand until its pilot passes its acceptance gate in §05.** All phases target the Entra groups built by `SASE-Set-PhaseScopeTargeting.ps1` (requirement R8).

> ***Per-phase preflight (every phase):*** *run* `SASE-Validation.ps1` *(readiness) and* `SASE-Report.ps1` *(capability-coverage + gap report) before Configure; confirm change control, rollback, and comms (R11). Use* `SASE-Remediation.ps1` *(default* `-WhatIf`*) to preview fixes for any readiness gaps.*

Phase 0 — Baseline & Steering

Goal: wire identity, deploy steering to a small scope, turn on **monitor-only** visibility.

1. **Assess.** Run `SASE-Discovery.ps1` (read-only) for the current-state edge/identity baseline (R6).

Confirm gating requirements R1–R2 are met.

1. **Configure.** Confirm Entra SSO (R3); deploy the edge client + steering config and the decryption

root certificate via endpoint management (R4) — but keep inspection/enforcement in monitor-only. Build the Phase 0 groups `SASE Phase Scope – Baseline – Pilot` / `– Broad`.

1. **Pilot.** Steer `SASE Phase Scope – Baseline – Pilot`; confirm traffic appears in the edge with correct identity attribution and no user impact (monitor-only).

1. **Validate.** Run the Phase 0 tests (§05). **Pilot-phase gate.**
2. **Expand.** Steer `SASE Phase Scope – Baseline – Broad`. Capture the KPI baseline (R12) now.

Phase 1 — Secure Web Gateway (SWG)

Goal: bring web/SaaS traffic under one SWG policy, with **SSL inspection** and the governed exception model.

1. **Assess.** Confirm certificate trust on the steered scope (R9); finalize the SSL-inspection exception list in the reference architecture.
1. **Configure.** Enable SSL inspection (honoring the exception list) and author SWG policy per the policy model — named objects, e.g. `SASE Policy – SWG Acceptable Use – Pilot`. (Exact console steps deferred to Netskope docs.)
1. **Pilot.** Apply to `SASE Phase Scope – SWG – Pilot`; verify inspection works (no TLS errors), policy decides as designed, exceptions are honored.
1. **Validate.** Run the SWG tests (§05). **Pilot-phase gate** — including zero broken pinned/regulated apps.
1. **Expand.** Promote policy to `SASE Phase Scope – SWG – Broad`; begin retiring the legacy SWG/proxy (policy-consistency KPI rises).

Phase 2 — CASB & Cloud-App Control

Goal: add sanctioned/unsanctioned cloud-app visibility and inline control; tame shadow IT.

1. **Assess.** Review cloud-app discovery from Phase 1 visibility; classify apps sanctioned/tolerated/unsanctioned.
1. **Configure.** Author CASB inline policy per the policy model (identity-first), e.g. `SASE Policy – CASB Cloud Control – Pilot`; connect any supported API-based posture for sanctioned apps. (Console specifics deferred to Netskope docs.)
1. **Pilot.** Apply to `SASE Phase Scope – CASB – Pilot`; verify shadow-IT visibility and that control actions behave as designed.
1. **Validate.** Run the CASB tests (§05). **Pilot-phase gate.**
2. **Expand.** Promote to `SASE Phase Scope – CASB – Broad`.

Phase 3 — ZTNA for Private Apps

Goal: publish private apps via identity-aware, per-app access; begin VPN displacement.

1. **Assess.** Confirm the private-app catalog mapped to Entra groups (R10).
2. **Configure.** Publish private apps via ZTNA, mapping each to its access group per the policy model, e.g. `SASE Policy – ZTNA Private App – Pilot`. (Connector placement / console steps deferred to Netskope docs.)
1. **Pilot.** Apply to `SASE Phase Scope – ZTNA – Pilot`; verify users reach exactly their entitled apps and nothing else; confirm the VPN path can be displaced for these users.

1. **Validate.** Run the ZTNA tests (§05). **Pilot-phase gate.**

2. **Expand.** Promote to SASE `Phase Scope – ZTNA – Broad`; retire broad VPN access as displacement

proves out (ZTNA/VPN-displacement KPI rises). Keep the VPN path warm per the rollback plan (R11) until displacement is validated.

Phase 4 — Data-Protection-Adjacent & Optimization

Goal: activate the designed-in content-aware hooks and tune the estate.

1. **Assess.** Review the DLP-adjacent hooks named in the reference architecture; confirm scope (this

blueprint *attaches* hooks; the deep DLP build is a separate program).

1. **Configure.** Activate the content-aware policy hooks where designed, e.g.

`SASE Policy – Data Hook – Pilot`; tune inspection exceptions, SWG/CASB policy, and performance.

1. **Pilot.** Apply to `SASE Phase Scope – DataHook – Pilot`; verify hooks fire as designed with no

undue user impact.

1. **Validate.** Run the optimization tests (§05). **Pilot-phase gate.**

2. **Expand.** Promote to `SASE Phase Scope – DataHook – Broad`; finalize the operating-model cadence

(Part B).

Part D — Transition to run

Once the roadmap phases pass:

1. Confirm all phase gates PASS (`SASE-Report.ps1` shows capability coverage with no critical gaps).

2. Hand the estate to the operating model (Part B): roles assigned, KPI dashboard live, review cadence

scheduled.

1. Keep the three design artifacts version-controlled; any change goes through the policy/exception

review and updates `SASE Reference Architecture – Edge – Design` / `SASE Policy Model – Edge – Design`.

Order of operations

`Design artifacts (A)` → `Operating model (B)` → `Phase 0 Baseline` → `Phase 1 SWG` → `Phase 2 CASB` → `Phase 3 ZTNA` → `Phase 4 Data-hook & Optimize` → `Run (D)` — each phase gated by its pilot.

5. Testing & Acceptance — SASE / SSE Architecture & Design Blueprint

Testing is organized by phase, each test with an explicit **PASS/FAIL gate**. Every phase has a **pilot acceptance gate** — the combined condition that must be met before the phase expands from pilot to broad scope (Implementation §03, Part C). **No phase expands until its pilot acceptance gate is PASS.**

> *Run* `SASE-Validation.ps1` *for the scripted readiness checks and* `SASE-Report.ps1` *for the capability-coverage + gap report; the manual gates below are operator-verified on a steered pilot endpoint. Record results against the phase, e.g.* `Validation Results – SWG – Pilot`*.*

Group 0 — Design & operating-model readiness (run once, before Phase 0 expands)

T0.1 — Design artifacts exist. `SASE Reference Architecture – Edge – Design`, `SASE Policy Model – Edge – Design`, and `SASE Adoption Roadmap – Edge – Program` are authored and approved (R7).

- PASS: all three approved. FAIL: any missing/unapproved.

T0.2 — Operating model defined. Roles in `SASE Operating Model – Edge – Run` are assigned and the KPI baseline is captured (R2, R12); `SASE-Report.ps1` produces a baseline.

- PASS: roles assigned + baseline captured. FAIL: no owner or no baseline.

T0.3 — Targeting groups present. Per-phase pilot/broad groups exist (R8); `SASE-Validation.ps1` confirms.

- PASS: groups present and correctly named. FAIL: missing/misnamed.

Readiness gate: T0.1–T0.3 PASS before Phase 0 expands.

Group 1 — Phase 0: Baseline & Steering

T1.1 — Steering active. Pilot endpoints in `SASE Phase Scope – Baseline – Pilot` are steered and appear in the edge.

- PASS: traffic visible from pilot endpoints. FAIL: not steered.

T1.2 — Identity attribution. Steered traffic is attributed to the correct user/Entra group.

- PASS: correct identity on traffic. FAIL: anonymous/wrong identity.

T1.3 — No user impact (monitor-only). Inspection/enforcement is monitor-only; users see no change.

- PASS: zero functional impact. FAIL: user-visible breakage.

Phase 0 acceptance gate: T1.1–T1.3 PASS for the pilot scope.

Group 2 — Phase 1: Secure Web Gateway (SWG)

T2.1 — Certificate trust. The decryption root is trusted on the steered scope; inspected sites load without TLS errors (R9).

- PASS: no TLS trust errors. FAIL: certificate warnings.

T2.2 — SSL inspection working. Inspectable HTTPS traffic is decrypted and visible to policy.

- PASS: inspected traffic visible. FAIL: traffic opaque/bypassed unexpectedly.

T2.3 — Exceptions honored. Destinations on the governed exception list are **not** decrypted; pinned/regulated apps work.

- PASS: exceptions bypass inspection, apps work. FAIL: a pinned/regulated app breaks.

T2.4 — SWG policy decides as designed. A test user hits allowed/blocked/inspected categories and gets the designed action per the policy model.

- PASS: actions match design. FAIL: wrong/inconsistent action.

SWG acceptance gate: T2.1–T2.4 PASS; **zero broken pinned/regulated apps**.

Group 3 — Phase 2: CASB & Cloud-App Control

T3.1 — Shadow-IT visibility. Unsanctioned cloud apps used by the pilot are discovered and listed.

- PASS: shadow IT visible. FAIL: no discovery.

T3.2 — Cloud-app control action. A sanctioned/tolerated/unsanctioned app gets its designed inline action (allow/restrict/block) per the policy model.

- PASS: control behaves as designed. FAIL: wrong action.

T3.3 — Identity-first targeting. The action correctly varies by the user's Entra group.

- PASS: per-group policy applies. FAIL: policy not identity-scoped.

CASB acceptance gate: T3.1–T3.3 PASS for the pilot scope.

Group 4 — Phase 3: ZTNA for Private Apps

T4.1 — Entitled access works. A pilot user reaches exactly the private apps mapped to their Entra group (R10).

- PASS: entitled apps reachable. FAIL: cannot reach an entitled app.

T4.2 — Least privilege enforced. The same user **cannot** reach private apps they are not entitled to (no broad network access).

- PASS: non-entitled apps blocked. FAIL: over-broad access.

T4.3 — VPN displacement. With ZTNA active for the pilot, the legacy VPN path is not required for the published apps.

- PASS: apps reachable without VPN. FAIL: still VPN-dependent.

ZTNA acceptance gate: T4.1–T4.3 PASS; rollback (re-enable VPN) confirmed available (R11).

Group 5 — Phase 4: Data-Protection-Adjacent & Optimization

T5.1 — Hooks fire. The designed-in content-aware hooks (`SASE Policy – Data Hook – Pilot`) trigger on the test condition.

- PASS: hook fires as designed. FAIL: no trigger / wrong trigger.

T5.2 — No undue impact. The hooks and tuned policy do not break normal pilot usage.

- PASS: usage unaffected. FAIL: false positives break work.

T5.3 — Performance acceptable. Inspection/steering overhead is within the agreed target on the pilot.

- PASS: performance within target. FAIL: unacceptable latency.

Optimization acceptance gate: T5.1–T5.3 PASS for the pilot scope.

Group 6 — Operating-model health (ongoing)

T6.1 — KPI dashboard live. `SASE-Report.ps1` produces current KPI values (steering/inspection coverage, policy consistency, ZTNA displacement, etc.).

- PASS: KPIs reported. FAIL: no measurement.

T6.2 — Exception register current. SSL-inspection and steering-bypass exceptions are documented and in-date.

- PASS: register current. FAIL: stale/undocumented exceptions.

Run gate: T6.1–T6.2 PASS before the program is declared in steady-state run.

Program-level exit (per phase)

A phase is **complete** only when its acceptance gate above is PASS. Capture results against the phase name (`Validation Results – <Phase> – Pilot`). **No phase expands to broad scope until its pilot gate passes** and change control + rollback are confirmed (R11).

6. Troubleshooting Guide — SASE / SSE Architecture & Design Blueprint

Failure modes and fixes, grouped by area. Most SASE/SSE problems trace back to a few root causes: **the decryption certificate isn't trusted**, **policy isn't keyed off identity**, **steering isn't active or has the wrong scope**, or **a phase was expanded before its pilot gate passed**.

> *Use* `SASE-Discovery.ps1` / `SASE-Validation.ps1` *(read-only) for current-state and readiness, and* `SASE-Report.ps1` *for capability coverage + gaps. Use* `SASE-Remediation.ps1` *(default* `-WhatIf`*) to preview fixes.* ***Exact Netskope console steps are deferred to the official Netskope documentation and your tenant*** *— these fixes name *what* to correct.*

A. Steering / client not active

Symptom	Cause	Remediation
Endpoint traffic not visible in the edge	Client/steering not deployed, or device outside the phase scope	Confirm the client + steering config deployed via endpoint management (R4); confirm the device is in SASE `Phase Scope – <Phase> – Pilot/Broad`; `SASE-Validation.ps1` checks scope membership
Steering active but no identity on traffic	SSO/identity not wired, or device not signed in with the Entra identity	Confirm Entra SSO to the edge (R3); confirm the user is signed in; check identity attribution in `SASE-Discovery.ps1` output
Only some users steered	Targeting group membership incomplete	Add members with `SASE-Set-PhaseScopeTargeting.ps1 -Apply`; re-check with `SASE-Validation.ps1`

B. SSL inspection / certificate trust

Symptom	Cause	Remediation
Browser TLS / certificate warnings on inspected sites	Decryption root not trusted on the endpoint	Deploy the root certificate via endpoint management (R4); confirm it is in the trusted store before enabling inline inspection (R9, T2.1)
A pinned-cert / regulated app breaks when inspection is on	App is being decrypted but does not tolerate inspection	Add the destination to the governed SSL-inspection **exception list** in `SASE Reference Architecture – Edge – Design`; confirm the exception is honored (T2.3)
Inspection coverage looks low	Too many exceptions, or traffic bypassing steering	Review the exception register (retire stale entries); confirm steering scope; track coverage/exception-rate via `SASE-Report.ps1` — do not chase 100%

C. Identity / policy not behaving as designed

Symptom	Cause	Remediation
Same action applied to everyone regardless of group	Policy not written identity-first	Rewrite per the policy model (identity → destination → content → action); target named Entra groups, not IP/all-users

Symptom	Cause	Remediation
Wrong allow/block decision	Policy order or destination matching incorrect	Verify the layered evaluation order in `SASE Policy Model – Edge – Design`; re-test the category/app (T2.4)
Policy inconsistent between SWG and CASB	Two policy logics drifted apart	Re-align both to the single `SASE Policy Model – Edge – Design`; the whole point is one model across services

D. CASB / cloud-app control

Symptom	Cause	Remediation
No shadow-IT discovery	Inline visibility not established (Phase 1) or traffic not steered	Confirm Phase 0/1 steering + inspection are active; cloud-app visibility rides the inline path (T3.1)
Cloud-app control action wrong	App not classified, or policy not identity-scoped	Classify sanctioned/tolerated/unsanctioned first; write identity-first CASB policy (T3.2–T3.3)

E. ZTNA / VPN displacement

Symptom	Cause	Remediation
User can't reach an entitled private app	App not published, or user not in the access group	Confirm the app is published and mapped to the user's Entra group (R10); re-check group membership; (connector specifics deferred to Netskope docs)
User can reach apps they shouldn't	Over-broad publishing (network-range instead of per-app/least-privilege)	Re-scope publishing to per-app + entitled group; verify non-entitled apps are unreachable (T4.2)
Removing VPN broke access	VPN displaced before ZTNA validated	Roll back per R11 (re-enable VPN path); validate ZTNA for the apps first (T4.1–T4.3), then displace

F. Phasing / gate violations

Symptom	Cause	Remediation
Broad rollout caused widespread breakage	A phase expanded before its pilot gate passed	Roll back the phase (R11); return to pilot scope; do not expand until the pilot acceptance gate is PASS (§05)

Symptom	Cause	Remediation
Coverage/gaps unclear mid-program	No coverage report being run	Run `SASE-Report.ps1` each phase preflight; track the gap matrix and KPIs

G. Design / operating-model issues

Symptom	Cause	Remediation
Program drifting back to point-tool config	No owner / artifacts not maintained	Assign SASE `Architecture Owner – Edge – Run` (R2); keep the three design artifacts current as living documents (Best Practices §07)
Exceptions accumulating unreviewed	No exception-review cadence	Institute the regular exception review in the operating model (§03 Part B3); re-justify or retire each (T6.2)
KPIs not measured	Operating model not stood up	Capture the baseline (R12) and keep `SASE-Report.ps1` running (T6.1)

H. Tooling / script issues

Symptom	Cause	Remediation
A script fails to connect to Graph	Microsoft Graph PowerShell module not installed or not consented	`Install-Module Microsoft.Graph -Scope CurrentUser`; sign in with the required read scopes when prompted
Remediation made no change	`-WhatIf` is the default (by design)	Re-run with `-Apply` once the preview is correct
Netskope-specific automation expected	Netskope module specifics are intentionally deferred	Use Netskope's official API/module documentation for tenant automation; this kit's scripts cover the Graph/identity/edge-readiness side

When to escalate

If steering is active and correctly scoped, the decryption root is trusted, policy is written identity-first per the policy model, the exception list is current, and a capability still misbehaves — collect the `SASE-Discovery.ps1` and `SASE-Validation.ps1` output and the relevant phase's `Validation Results – <Phase> – Pilot`, and open a support case with your Netskope vendor referencing the

specific capability and policy object.

7. Best Practices — SASE / SSE Architecture & Design Blueprint

Program-grade practices for designing and adopting a unified SASE/SSE edge. These reduce risk, prevent the slide back into piecemeal tooling, and keep the estate operable over time.

Design discipline

- **Design before you buy.** Author the reference architecture, policy model, and roadmap *first*

(`SASE Reference Architecture – Edge – Design` and friends). Designing first is what exposes duplicate tools and overlapping paths before more spend is committed.

- **Make identity the control plane, everywhere.** Decide policy from identity + device posture, not

network location — and wire it the *same* way into SWG, ZTNA, and CASB. One trust model is the whole point of SASE/SSE.

- **One policy model, applied consistently.** Write every policy in the layered order — identity →

destination/app → content hook → action. Inconsistent policy logic across consoles is how the old mess returns.

- **Name everything with the convention.** `<Purpose> – <Scope> – <Stage>`, reused verbatim across

artifacts, groups, phases, and scripts, so discovery, validation, reporting, and policy cross- reference cleanly.

Phasing & the pilot gate

- **Adopt in phases, never big-bang.** Phase 0 baseline → SWG → CASB → ZTNA → data-hook/optimize. Each

phase changes a bounded slice of live traffic.

- **The pilot gate is non-negotiable.** No phase expands to broad scope until its pilot acceptance gate

(§05) is PASS. A bounded pilot catches breakage before the fleet feels it.

- **Keep cohorts intact in a phase scope.** Scope a phase to groups of users who work together so

coexistence between unified and legacy paths is short.

- **Run the per-phase preflight.** `SASE-Validation.ps1` (readiness) and `SASE-Report.ps1` (coverage +

gaps) before every Configure; preview fixes with `SASE-Remediation.ps1` (default `-WhatIf`).

SSL inspection

- **Trust the certificate first.** Deploy and confirm the decryption root on the steered scope *before*

enabling inline inspection — otherwise users get TLS errors everywhere.

- **Govern the exception list; don't chase 100%.** Inspect by default, but maintain a documented, reviewed exception list for pinned-cert, regulated, and privacy-sensitive traffic. Inspection coverage is a tracked KPI with an exception rate, not a number to force to 100%.
- **Review exceptions on a cadence.** Every exception is re-justified or retired; stale bypasses are how blind spots accumulate.

ZTNA & VPN displacement

- **Least privilege, per app.** Publish private apps mapped to the entitled Entra group — not broad network ranges. Verify non-entitled apps are unreachable (T4.2).
- **Displace VPN, don't yank it.** Prove ZTNA works for the pilot's apps, then retire broad VPN as displacement validates — and keep the VPN path warm for rollback (R11) until it does.

CASB & shadow IT

- **Classify before you control.** Use Phase 1 visibility to label apps sanctioned/tolerated/unsanctioned, then write inline policy — control without classification breaks legitimate work.
- **Keep CASB policy identity-first** like everything else, so the same app can be handled differently per group.

Data-protection-adjacent scope

- **Design the hooks, defer the DLP build.** Mark where content-aware policy attaches now; building a full data-classification/DLP program is a separate engagement. Naming the hooks keeps that later work a drop-in, not a rebuild.

Change control & rollback

- **Approved window + tested back-out per phase (R11).** Every phase changes live traffic; have a documented rollback (disable steering / remove client / revert policy / re-enable VPN) and a comms plan before each cutover.
- **Communicate the change.** A short note per phase — what changes, the new sign-in/access experience, and who to contact — sharply reduces tickets.

Operating model

- **Assign the roles before you expand.** SASE `Architecture Owner`, `Policy Administrator`, `Operations Engineer`, `Identity Liaison`, and the executive `Sponsor` — a design with no owner decays.

• **Measure what matters.** Keep the KPI dashboard (`SASE-Report.ps1`) live: steering coverage, inspection coverage/exception rate, policy consistency, ZTNA/VPN displacement, mean time to policy change, user-impact rate.

• **Keep the artifacts current as you go.** The reference architecture, policy model, and roadmap are living documents; update them with every approved change, not at the end.

8. Deliverables Summary — SASE / SSE Architecture & Design Blueprint

This package is a flagship, enterprise-grade design kit for building a unified SASE/SSE edge. It contains the documentation blueprint, five PowerShell tools, and a hero image. It delivers a reference architecture, a policy model, a phased adoption roadmap, and a target operating model — and the read-only tooling to discover, validate, report, remediate, and scope the rollout.

Documentation

File	What it covers
`executive-summary.md`	One-page business case and outcomes for sponsors/leadership.
`01-solution-overview.md`	What the blueprint is, *why* piecemeal edge security fails, the capability services (SWG, ZTNA, CASB, SSL inspection, DLP-adjacent), identity as the control plane, the phased roadmap, the operating model, benefits, and honest limitations.
`02-requirements.md`	R1–R12 prerequisites (Description / Why / Verify / Implement-if-missing), including the GATING Netskope-tenant/modules and executive-sponsorship requirements, plus a checklist.
`global-requirements-network.md`	Category-wide Network prerequisites (Netskope tenant, Entra ID, Intune, admin roles, network/identity fundamentals, pilot group, change control), published once.
`03-implementation.md`	The program runbook: design artifacts (reference architecture, policy model, roadmap), the target operating model (roles/RACI, KPIs, cadence), and the phased adoption roadmap (Phase 0 Baseline → SWG → CASB → ZTNA → Data-hook/Optimize) with a pilot-phase gate per phase.
`05-testing.md`	Per-phase numbered tests with PASS/FAIL gates and a pilot acceptance gate per phase, plus design-readiness and operating-model-health gates.

File	What it covers
`06-troubleshooting.md`	Field failure modes and fixes across steering/client, SSL inspection/certificates, identity/policy, CASB, ZTNA/VPN displacement, design/operating-model, and tooling.
`07-best-practices.md`	Field-proven practices: design discipline, phasing & the pilot gate, SSL inspection, ZTNA/VPN displacement, CASB, DLP-adjacent scope, change control, and the operating model.

> ***Vendor scope:*** *the exact* ***Netskope console*** *configuration is* ***deferred to the official Netskope documentation and your tenant****. This kit is authored at the architectural and procedural level — what to design, in what order, with which named artifacts and gates.*

PowerShell scripts

All scripts carry a header with **PURPOSE / HOW TO RUN / EXPECTED RESULTS** and a neutral, white-label note. Current-state edge/identity discovery uses **Microsoft Graph PowerShell**; Netskope-module specifics are flagged and deferred to Netskope's documentation. Discovery, Validation, and Report are strictly **read-only**; Remediation defaults to `-WhatIf`.

Script	What it does
`scripts\SASE-Discovery.ps1`	**Read-only current-state inventory.** Uses Microsoft Graph to capture the edge/identity baseline (R6): tenant identity, targeting groups, enrolled/compliant devices, conditional-access signals, and notes for non-discoverable third-party SWG/VPN/CASB. The design baseline.
`scripts\SASE-Validation.ps1`	**Architecture-readiness validation (read-only).** Checks SSO/identity readiness, the presence and naming of the per-phase targeting groups, device/certificate-trust readiness signals, and emits PASS/FAIL per check for the readiness and per-phase gates.
`scripts\SASE-Report.ps1`	**Capability-coverage + gap report (read-only).** Produces the operating-model KPI view and a coverage/gap matrix across the capability services and roadmap phases, as a CSV plus a console summary — the run dashboard.
`scripts\SASE-Remediation.ps1`	**Safe remediation of configuration gaps.** Fixes readiness gaps the validation/report surface (e.g. create missing targeting groups, add members, tag device groups). Runs `-WhatIf` by default; requires `-Apply` to change anything.

Script	What it does
`scripts\SASE-Set-PhaseScopeTargeting.ps1`	**Phase-scope group builder.** Creates/maintains the named Entra security groups that scope each roadmap phase and its pilot (`SASE Phase Scope – <Phase> – Pilot` / `– Broad`), per the naming convention. Defaults to `-WhatIf`.

Hero image

File	Purpose
`SASE-hero.png`	Listing/marketing hero image for the solution.

What the buyer receives

- This blueprint as a branded **Word** document and **PDF** (`SASE-Solution.pdf`).
- All five **PowerShell scripts** in a single ZIP (`SASE-Scripts.zip`).
- A repeatable design + adoption process following the standard model:

Assess → **Configure** → **Pilot** → **Validate** → **Expand**, gated per phase.

> ***Naming convention reminder:*** *artifacts, phases, roles, groups, and script outputs share the standard* `<Purpose> – <Scope> – <Stage>` *(e.g.* `SASE Phase Scope – SWG – Pilot`*) so everything cross-references cleanly across discovery, validation, reporting, and policy.*

Outcome

One coherent SASE/SSE architecture — SWG, ZTNA, CASB, SSL inspection, and identity-based policy designed as a single operating model with a phased roadmap — replacing piecemeal edge security, with a named operating model and measurable KPIs, proven on a pilot phase before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SASE-Discovery.ps1

SASE-Remediation.ps1

SASE-Report.ps1

SASE-Set-PhaseScopeTargeting.ps1

SASE-Validation.ps1